

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RESISTENCIA

Aplicación Web y Servicio Redis Contenerizados

DEVOPS

Trabajo Práctico 1

Integrantes

- Sixto Feliciano **Arrejin**.
- Tobias Alejandro **Maciel Meister**.
- André Leandro **San Lorenzo**.

29 de septiembre de 2025

Índice

1. Introducción.....	1
2. Desarrollo.....	1
Selección de Tecnologías.....	1
Estructura del Proyecto.....	1
Repositorio GitHub.....	2
Desarrollo de la Lógica de la Aplicación.....	2
Contenerización con Docker.....	4
3. Resultados Obtenidos.....	8
4. Dificultades Encontradas.....	8
5. Posibles Mejoras.....	9
6. Conclusión.....	9
7. Fuentes.....	10

1. Introducción

El presente informe detalla el desarrollo de la aplicación web SmartCatalog, un sistema diseñado para evaluar y comparar descripciones de productos generadas por distintos modelos de inteligencia artificial generativa.

Esta aplicación surgió como una propuesta de uno de los integrantes del equipo para un proyecto de trabajo, lo que añadió un grado extra de complejidad: no se trataba de una aplicación creada exclusivamente con fines académicos, sino de una herramienta con un caso de uso real. Esto implicó mayores desafíos técnicos durante el desarrollo, pero al mismo tiempo brindó más oportunidades de aprendizaje y aplicación práctica de conceptos avanzados.

La solución fue contenerizada con Docker y desplegada en Microsoft Azure, integrando un sistema de caché con Redis y una base de datos PostgreSQL gestionada a través de Supabase. Además, se implementó un pipeline CI/CD automatizado mediante GitHub Actions, lo que permitió aplicar de forma práctica principios y prácticas de DevOps en la integración y despliegue de servicios contenerizados.

2. Desarrollo

Selección de Tecnologías

La aplicación fue desarrollada utilizando las siguientes tecnologías:

- Frontend: Next.js + React + TailwindCSS
- Backend: Flask + Redis + PostgreSQL
- Containerización: Docker + Docker Compose
- CI/CD: GitHub Actions
- Cloud: Microsoft Azure

Estructura del Proyecto

El proyecto se organizó en tres carpetas principales:

- Backend (Flask): Implementa la API para la gestión de productos, votaciones y carga de archivos CSV (no implementada en su totalidad). Incluye servicios de caché con Redis y almacenamiento en PostgreSQL.
- Frontend (Next.js + React): Proporciona la interfaz web para los usuarios, permitiendo votar, comparar descripciones y gestionar datos.

- Github: Donde se configura las pipelines de CI/CD para GitHub Actions.

También se incluyen archivos de configuración para la contenerización como lo son el Dockerfile y el docker-compose.yml.

Repositorio GitHub

El código fuente del proyecto se encuentra disponible en el siguiente repositorio:

- <https://github.com/Andre-Leandro/Description-Evaluator/tree/docker>

Dado que este proyecto se construyó sobre un trabajo previo de uno de los integrantes, para este práctico trabajamos específicamente sobre la rama *docker*, en la cual se configuraron los Dockerfiles, el archivo docker-compose.yml y los flujos de trabajo de GitHub Actions.

Desarrollo de la Lógica de la Aplicación

Backend

El backend implementa endpoints RESTful que permiten:

- Consultar productos y descripciones.
- Registrar votos de usuarios para diferentes modelos.
- Caché de las consultas mediante redis.

El sistema incorpora Redis como mecanismo de caché con el objetivo de optimizar el rendimiento y reducir la latencia en las consultas que se ejecutan con mayor frecuencia. Redis, al ser una base de datos en memoria de alta velocidad, permite responder en milisegundos a peticiones que de otro modo requerirían acceder directamente a la base de datos relacional, lo cual implicaría un mayor tiempo de respuesta.

Para la persistencia de datos se utiliza PostgreSQL, gestionado a través de Supabase, que actúa como base de datos principal encargada de almacenar de manera confiable la información crítica del sistema, como los datos de productos y las evaluaciones realizadas por los usuarios.

El sistema sigue el patrón de caché conocido como *cache-aside*. En este enfoque, la aplicación es la responsable de decidir cuándo leer y escribir en la caché. Su funcionamiento puede describirse en dos pasos principales:

Lectura:

- La aplicación primero consulta la caché (Redis).
- Si los datos están allí (*cache hit*), se devuelven inmediatamente con muy baja latencia.
- Si no se encuentran (*cache miss*), la aplicación obtiene los datos desde PostgreSQL y luego los almacena en Redis para futuras solicitudes.

Escritura/actualización:

- Cuando la aplicación modifica datos en PostgreSQL, invalida o actualiza manualmente la entrada correspondiente en Redis, asegurando que la caché no quede desincronizada.

Frontend

En el frontend se implementó un sistema de votación intuitivo y fácil de usar, acompañado de una visualización de resultados en tiempo real, lo que permite a los usuarios seguir de manera dinámica la evolución de las preferencias.

Asimismo, se incorporó la funcionalidad de carga y descarga de datos en formato CSV. Sin embargo, estas funcionalidades no se encuentran integradas con el backend, ya que el proyecto fue desarrollado en el marco de una POC (Prueba de Concepto), donde el foco principal estuvo en validar la experiencia de uso y la interacción básica del sistema.

Para la gestión de estilos y el diseño de la interfaz se utilizó TailwindCSS, lo que permitió mantener una estética consistente, moderna y responsiva, reduciendo la complejidad en la personalización de componentes visuales.

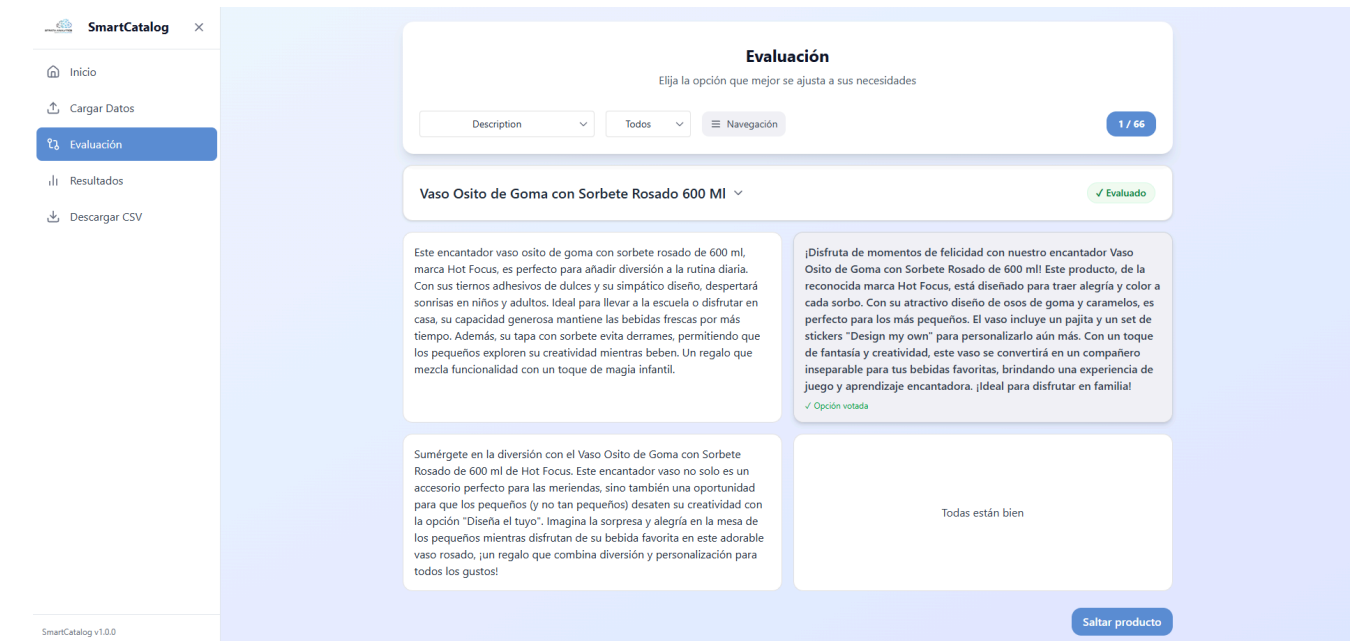


Figura 1: Vista de evaluación de descripciones.



Figura 2: Vista de estadísticas de votaciones.

Contenerización con Docker

Se crearon Dockerfiles independientes para el backend y el frontend, lo que permite construir imágenes ligeras, reproducibles y fáciles de mantener. Cada servicio cuenta con su propia definición de dependencias, configuraciones y pasos de compilación,

garantizando un entorno de ejecución consistente en todas las etapas (desarrollo, pruebas y producción).

El servicio de Redis se desplegó utilizando la imagen oficial *redis:alpine*, una versión optimizada y de bajo peso, ideal para entornos contenerizados. Esto asegura un rendimiento adecuado en la gestión de la caché con un consumo mínimo de recursos.

Para la orquestación de los servicios se configuró un archivo *docker-compose.yml*, encargado de levantar y coordinar los tres contenedores principales (backend, frontend y Redis) en un único entorno. Esto facilita la ejecución del sistema completo de forma local, simplificando las tareas de desarrollo, pruebas integradas y depuración.

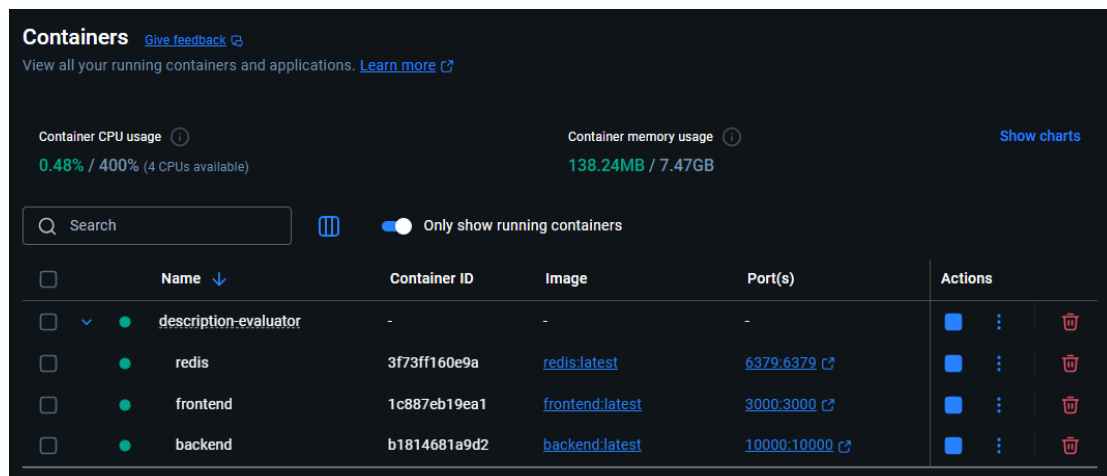


Figura 3: Contenedores en ejecución orquestados por el *docker-compose*.

Azure

En el entorno de producción, las imágenes se publican en Azure Container Registry (ACR) y se despliegan en Azure Container Instances (ACI), lo que permite escalar los servicios de forma modular y mantener una separación clara entre los componentes.

El ciclo de despliegue está respaldado por un pipeline CI/CD automatizado en GitHub Actions, el cual gestiona las etapas de:

1. Build de imágenes a partir de los Dockerfiles.
2. Push al registro privado de Azure (ACR).
3. Deployment automático en las instancias de contenedores (ACI).

De esta forma, se garantiza un proceso automatizado, repetible y confiable, alineado con las prácticas de DevOps y la integración de servicios contenerizados en entornos cloud.

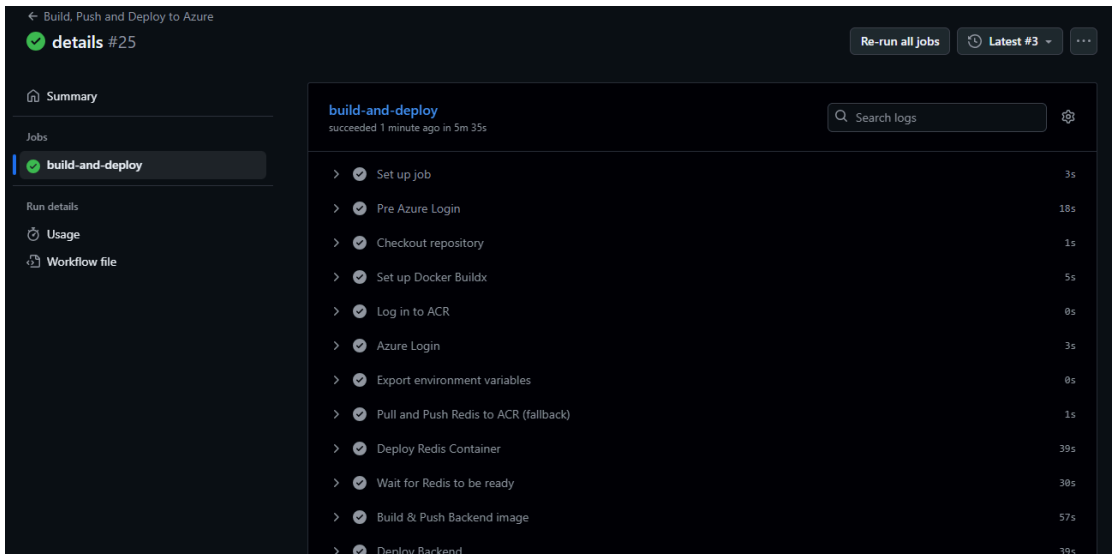


Figura 4: Pipeline *build-and-deploy* en GitHub Actions que gestiona despliegue en Azure.

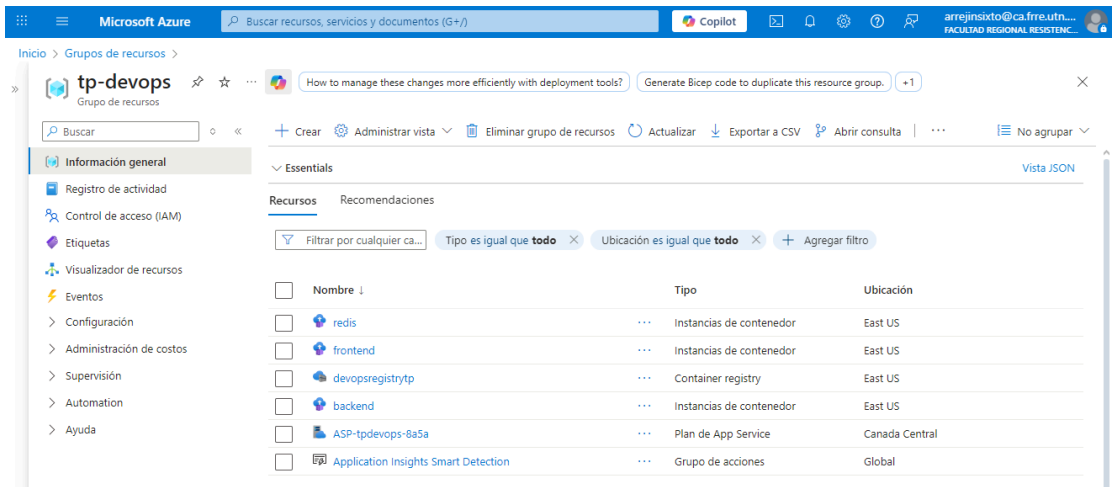


Figura 5: Grupo de recursos tp-devops.

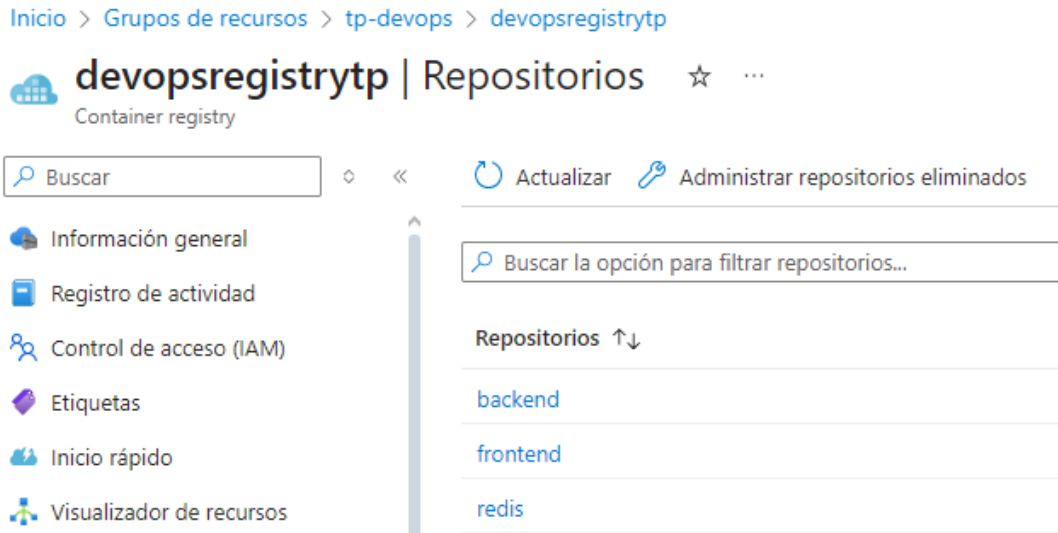


Figura 6: Container registry con las imágenes alojadas.

Inicio > **Instancias de contenedor** ...

Facultad Regional Resistencia (ca.fre.utn.edu.ar)

+ Crear ⚙ Administrar vista Actualizar Exportar a CSV Abrir consulta Asignar etiquetas No

Está viendo una nueva versión de la experiencia de exploración. Haga clic aquí para acceder a la experiencia anterior.

Filtrar por cualquier ca... Suscripción es igual que **todo** Grupo de recursos es igual que **tp-devops (3)** Ubicación es igual que **todo** + Agregar filtro

☐	Nombre ↑	Grupo de recursos	Ubicación	Estado	Tipo de sistema op...	Total de contenedo...	Suscripción
☐	backend	tp-devops	East US	En ejecución	Linux	1	Azure for Students
☐	frontend	tp-devops	East US	En ejecución	Linux	1	Azure for Students
☐	redis	tp-devops	East US	Creando	Linux	1	Azure for Students

Figura 7: Instancias de contenedor en ejecución.

frontend ...

Instancias de contenedor

Buscar

Información general

Registro de actividad

Control de acceso (IAM)

Etiquetas

Visualizador de recursos

Configuración

Contenedores

Identidad

Propiedades

Bloqueos

Supervisión

Métricas

Alertas

Registros

Información esencial

Grupo de recursos (mover) [tp-devops](#)

SKU Standard

Estado En ejecución

Tipo de SO Linux

Ubicación East US

Dirección IP (Public) 57.151.16.191

Suscripción (mover) [Azure for Students](#)

FQDN frontend-tp-devops.eastus.azurecontainer.io

Id. de suscripción 811d5445-f311-40c1-be02-5f92a5819ee2

Recuento de contenedores 1

Etiquetas (editar) [Agregar etiquetas](#)

CPU (millinúcleos)

12

10

8

Figura 8: Información de la instancia de contenedor frontend.

3. Resultados Obtenidos

Según nuestra percepción, el proyecto alcanzó con éxito los objetivos planteados en la consigna, logrando integrar y utilizar una amplia variedad de tecnologías ya conocidas por los integrantes del equipo, así como también la incorporación de nuevas herramientas en el pipeline de despliegue de contenedores.

En términos funcionales, se obtuvo:

- **Aplicación web con su API operativa**, integrando Redis como caché bajo el patrón *cache-aside* para optimizar el rendimiento en consultas frecuentes. Asimismo, se logró la contenerización y orquestación completa de los servicios mediante Docker y Docker Compose, y se configuró un pipeline de CI/CD con GitHub Actions para automatizar tanto la publicación de las imágenes en Azure Container Registry como el despliegue de las mismas en ACI.
- **Repositorio documentado y versionado**: El código fuente se mantuvo en un único repositorio de GitHub. Esta decisión permitió gestionar los cambios de manera ordenada, versionar correctamente y centralizar las configuraciones de CI/CD y contenedores.

4. Dificultades Encontradas

- Configuración inicial de contenedores y redes en Docker Compose. Inicialmente se presentaron algunas dificultades para hacer que los 3 contenedores (backend, frontend y redis) se comunicaran entre sí. Logramos levantar los 3 pero no tenían comunicación entre sí.
- Al momento de realizar el despliegue en Azure, surgieron nuevas dificultades relacionadas con la comunicación entre contenedores. En el entorno local, esta integración se resolvía fácilmente a través de Docker Compose, pero dicha solución sólo era válida en ese contexto. En el entorno cloud fue necesario investigar y aplicar alternativas específicas para Azure, adaptando la configuración de red y los endpoints de los servicios para asegurar una correcta interacción entre los contenedores desplegados.
- Configuración de GitHub Actions para la autenticación con Azure. Desde el inicio se pretendía hacer el despliegue directamente en Azure, sin embargo se presentaron algunas dificultades a la hora de iniciar sesión en el workflow hacia Azure, por lo que se consideraba y se llegó a realizar el deploy en Render.

Para resolver la dificultad de autenticación entre el workflow y Azure, se utilizó la consola de Azure (Azure CLI) para crear una Service Principal (entidad de servicio). Esta es una identidad de aplicación que permite a GitHub Actions autenticarse de forma segura en nuestra suscripción sin necesidad de usar credenciales personales.

Se ejecutó el siguiente comando en a través de la shell de Azure:

```
az ad sp create-for-rbac --name "tp-devops-sp" --role contributor  
--scopes /subscriptions/<SUSCRPTION-ID>/resourceGroups/tp-devops  
--sdk-auth
```

Este comando generó un objeto JSON con las credenciales necesarias (clientId, clientSecret, tenantId y subscriptionId). Este bloque JSON se configuró como un secreto (AZURE_CREDENTIALS) en el repositorio de GitHub, permitiendo que el workflow finalmente pudiera iniciar sesión en Azure y desplegar los contenedores de forma automatizada

Posteriormente, sorteadas estas dificultades pudimos implementar el deploy en Azure como se planteó inicialmente.

5. Posibles Mejoras

- Migrar la orquestación a Kubernetes.
- Incorporar monitoreo con Grafana o Azure Monitor con alertas configuradas para detectar fallas o caídas de servicios en tiempo real.
- Usar una arquitectura de Redis de High Availability.

6. Conclusión

Con este trabajo logramos alcanzar el objetivo de poner en práctica el uso de contenedores e integrar distintos servicios en una misma aplicación de forma coherente y escalable.

La decisión de utilizar Docker y Docker Compose resultó clave, ya que permitió aislar cada servicio, definir entornos reproducibles y simplificar el despliegue. Esta estrategia no solo facilitó la colaboración dentro del equipo, sino que también redujo problemas de compatibilidad, algo crítico cuando se combinan tecnologías y sistemas operativos diferentes.

Sobre esa base, se configuró un pipeline CI/CD con GitHub Actions, encargado de automatizar la construcción, prueba y publicación de las imágenes en la nube. Esta práctica eliminó pasos manuales propensos a errores, permitió ciclos de entrega más

cortos y sentó las bases para un flujo de integración continua alineado con buenas prácticas de DevOps.

Durante el proceso aparecieron dificultades, principalmente en la configuración de contenedores y la comunicación entre servicios, que requirieron iteraciones, depuración y validación. Resolver estos problemas no solo aportó aprendizaje técnico concreto, sino que también nos permitió ejercitar habilidades de análisis y resolución de incidentes, competencias esenciales en entornos reales de desarrollo y operaciones.

7. Fuentes

- Docker. (s. f.). *Docker documentation*. Docker. <https://docs.docker.com>
- Redis. (s. f.). *Redis documentation*. Redis. <https://redis.io/documentation>
- Flask. (s. f.). *Flask best practices for Docker*. Flask. <https://flask.palletsprojects.com/en/latest/deploying/docker/>
- Redis. (s. f.). *Highly Available Redis*. Redis. <https://redis.io/technology/highly-available-redis/>